9. Intent Inference + Multimodal Visual Understanding + SLAM Navigation

1. Course Content

Note: This chapter requires completing the map mapping file configuration according to the 【Multimodal Visual Understanding + SLAM Navigation】 chapter.

1. Learn to use the RAG knowledge base to train personal intent understanding.

Course review:

If you need to enhance or reduce the diversity of the large model's replies, refer to 【AI Large Model Development】-【RAG Knowledge Base Deployment】 to modify the knowledge base and sample library.

Using the RAG knowledge base can expand the knowledge scope of the large model. This chapter explains how to extend the RAG knowledge base so that the large model can understand personal intents.

Special note!!!:

1. The personal intent understanding function varies by user, and large model replies are divergent, so the actual debugging results are not exactly the same.
2. Extending the personal intent understanding function must comply with social morality, public order, and laws and regulations. The technical service is not responsible for the final debugging results of this chapter or their impact.

2. Preparation

2.1 Content Description

> Add a custom intent inference knowledge base. Since personal intents are divergent, there is no unified content, and the actual debugging results are not exactly the same.
>
> The virtual machine used by default with this product starts the Dify Docker environment automatically at boot, so no additional startup is usually needed.

Open the virtual machine terminal, start the Dify environment:

```
sh ~/bringup_dify.sh
```

```
yahboom@yahboom:~$ sh ~/bringup_dify.sh
[+] up 13/13
 ✔ Container docker-sandbox-1          Running                    0.0s
 ✔ Container docker-web-1              Running                    0.0s
 ✔ Container docker-db_postgres-1      Healthy                   22.5s
 ✔ Container docker-ssrf_proxy-1       Running                    0.0s
 ✔ Container docker-redis-1            Running                    0.0s
 ✔ Container docker-weaviate-1         Running                    0.0s
 ✔ Container docker-worker-1           Running                    0.0s
 ✔ Container docker-api_websocket-1    Running                    0.0s
 ✔ Container docker-plugin_daemon-1    Running                    0.0s
 ✔ Container docker-worker_beat-1      Running                    0.0s
 ✔ Container docker-api-1              Running                    0.0s
 ✔ Container docker-nginx-1            Running                    0.0s
 ✔ Container docker-init_permissions-1 Exited                    28.4s
yahboom@yahboom:~$
```

3. Configuration Documentation

3.1 Preparing the xlsx Document

Open the 【Intent Mapping】 folder in this chapter's folder, and select the corresponding camera 【Intent Mapping_xx.xlsx】 file according to your car configuration.

decision-layer-training-examples-en.xlsx 2026/8/14 11:58 XLSX 工作表 6 KB

query	answer
Go to point A	1. Navigate to point A
Go to point A and take a look	1. Navigate to point A 2. Capture the current view 3. Describe what is visible and end the task
Go to points A and B in sequence and see what is there	1. Navigate to point A 2. Capture and describe the current view at point A 3. Navigate to point B 4. Capture and describe the current view at point B 5. End the task
Return to the starting position	1. Navigate back to the starting position
Where are you now?	1. Get the current position 2. Reply with the current position and end the task
Record your current position	1. Record the current position 2. Reply with the result and end the task
Move forward 1 meter	1. Move forward 1 meter
Turn left 90 degrees	1. Turn left 90 degrees
Move forward 1 meter, then turn left 90 degrees	1. Move forward 1 meter 2. Turn left 90 degrees
Stop	1. Stop the current action
Point the camera straight ahead	1. Center the camera
Look left with the camera	1. Turn the camera to the left
See what is in front of you	1. Capture the current view 2. Describe what is ahead and end the task
Observe the surroundings	1. Observe the surroundings 2. Summarize the surroundings and end the task
Is there anything red in front of you?	1. Capture the current view 2. Determine whether a red object is present and report the result
See what is on the left	1. Turn the camera to the left 2. Capture the current view

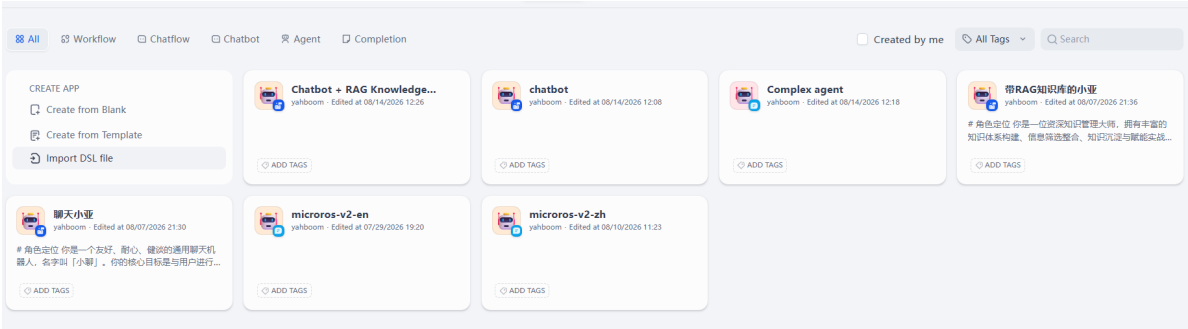
(You can add multiple files by yourself for more customized information.)

【Intent Mapping】: stores personal intents and the operations the user expects the large model to control the robot to perform.

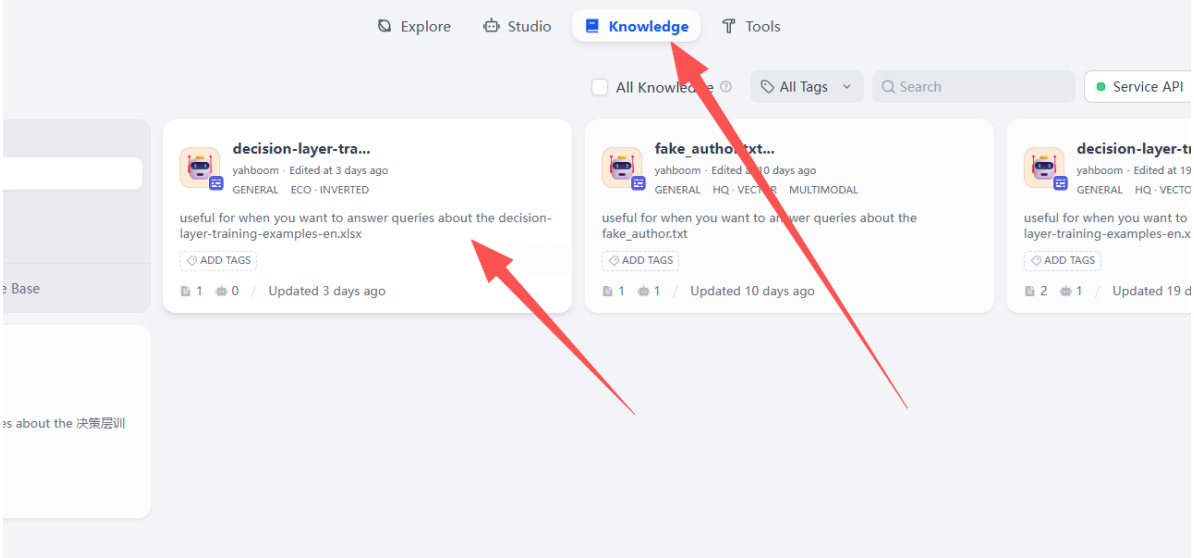
3.2 Uploading to the RAG Knowledge Base

Some intent inference is already configured at the factory. The following steps are for adding your own new intents.

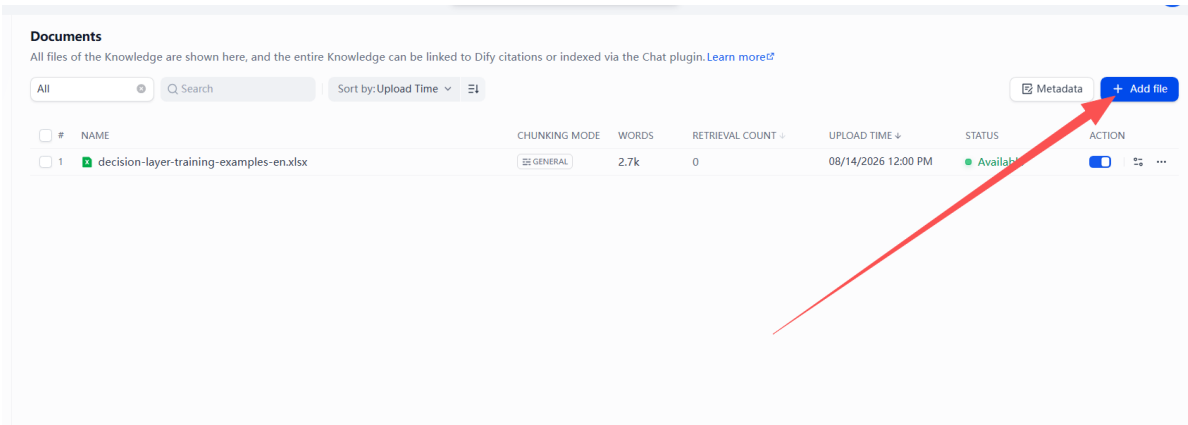
1. Access the IP of the vehicle and enter the Dify backend.



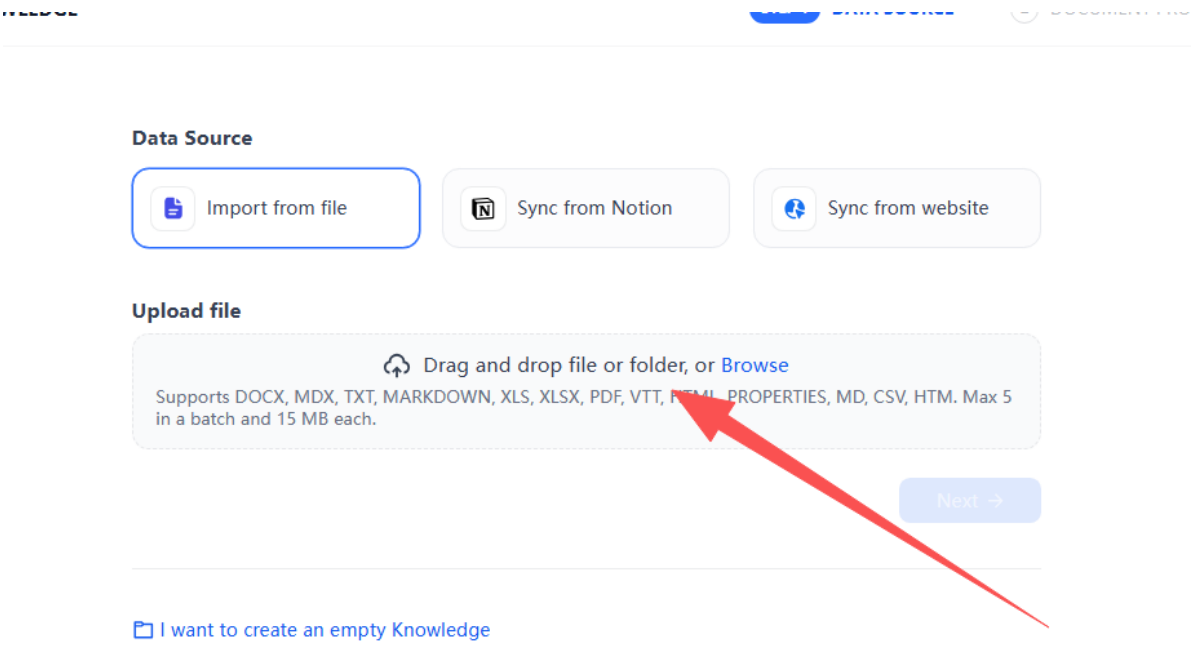
2. Click 【Knowledge】 and click 【Sample Library】.



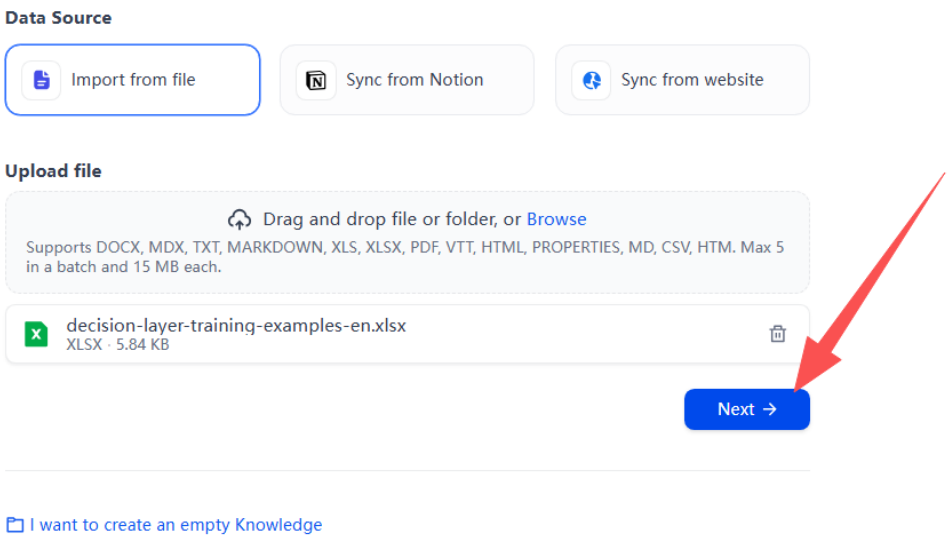
3. Click "Add File".

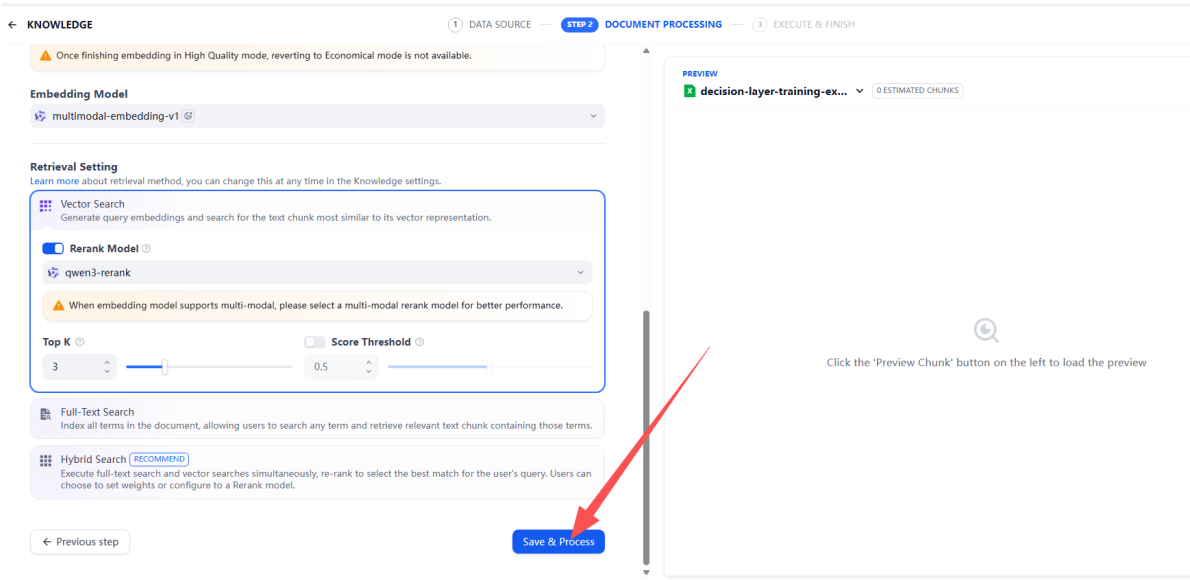


4. Upload your intent inference knowledge base.

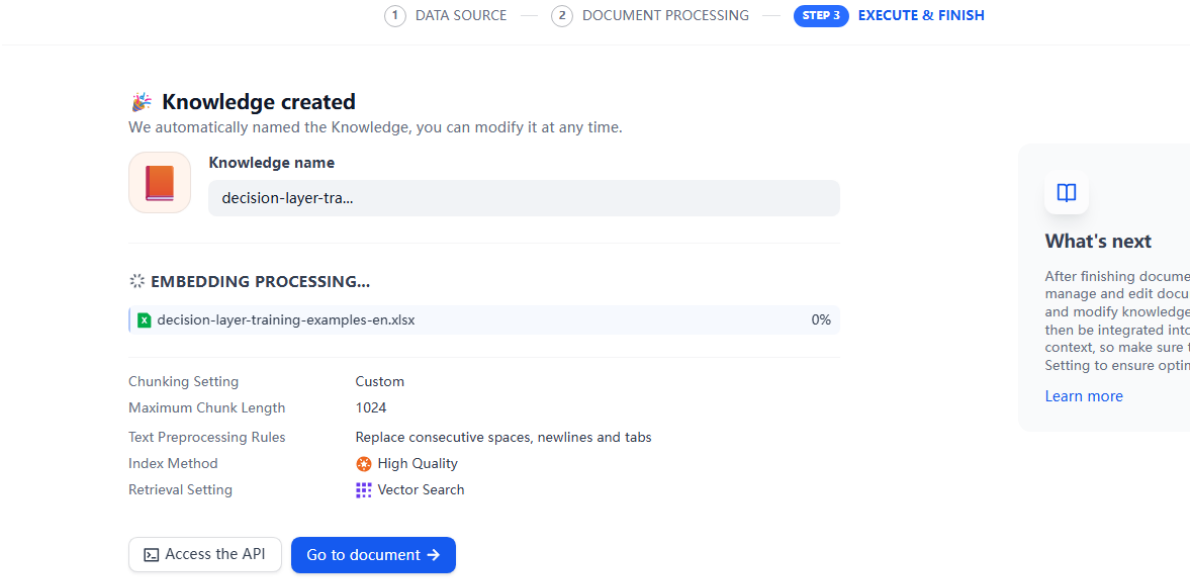


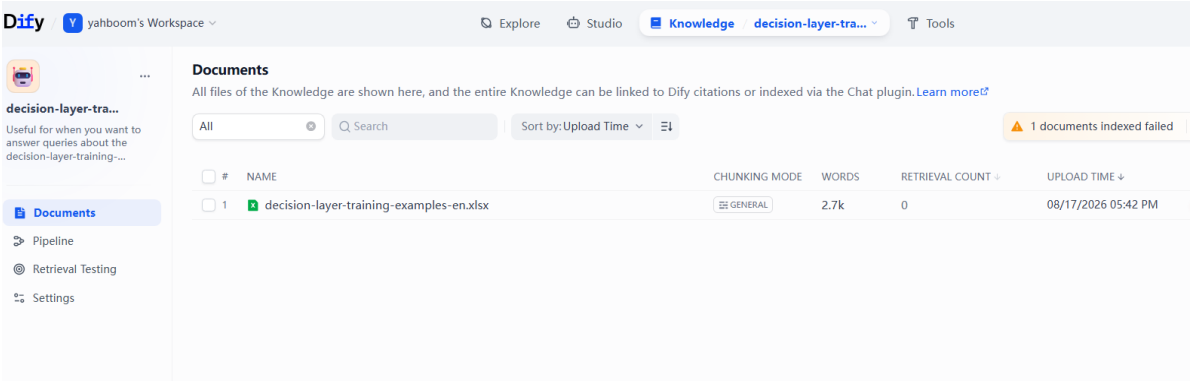
5. Click 【Next】, then click 【Save and Process】.



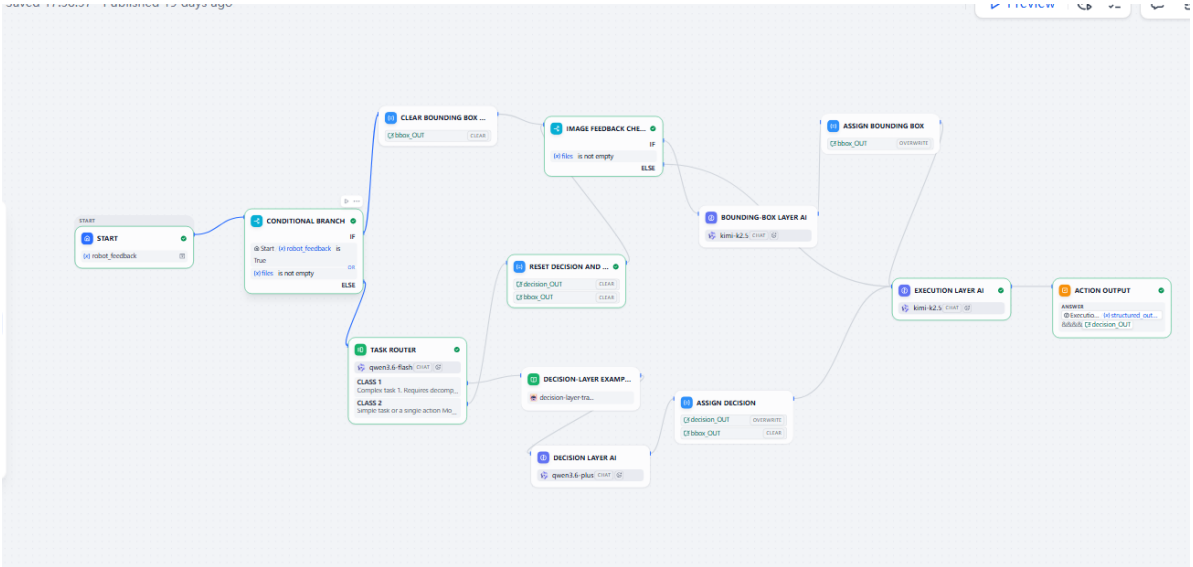


6. Wait for the processing to succeed, refresh the knowledge base, and you can see that the intent mapping file has been added.

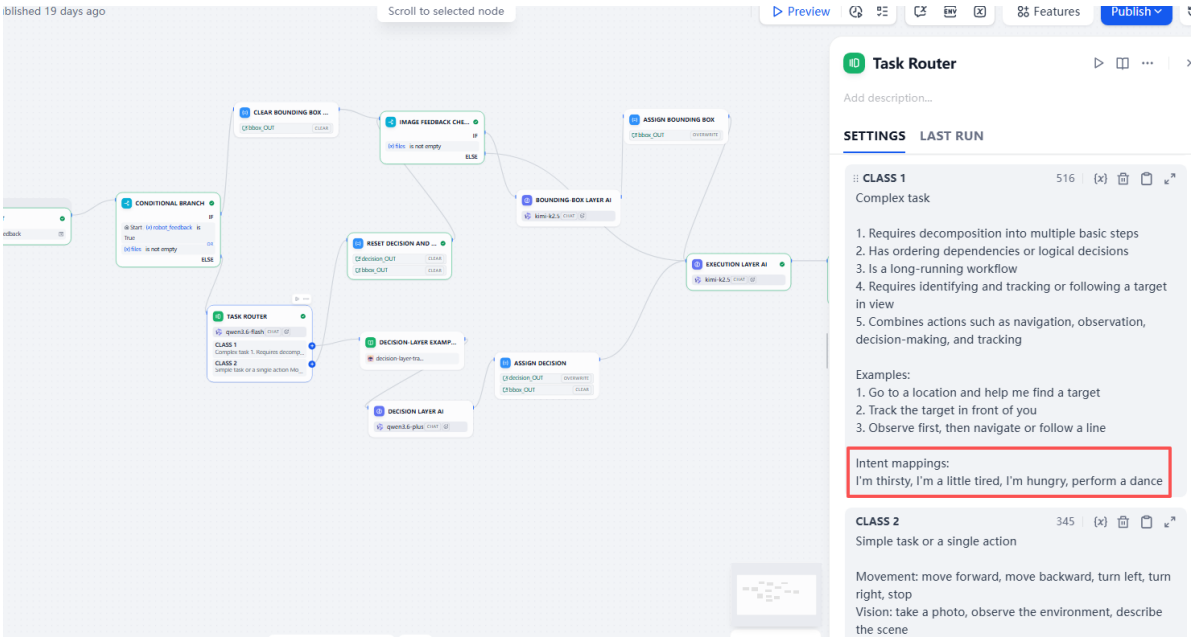




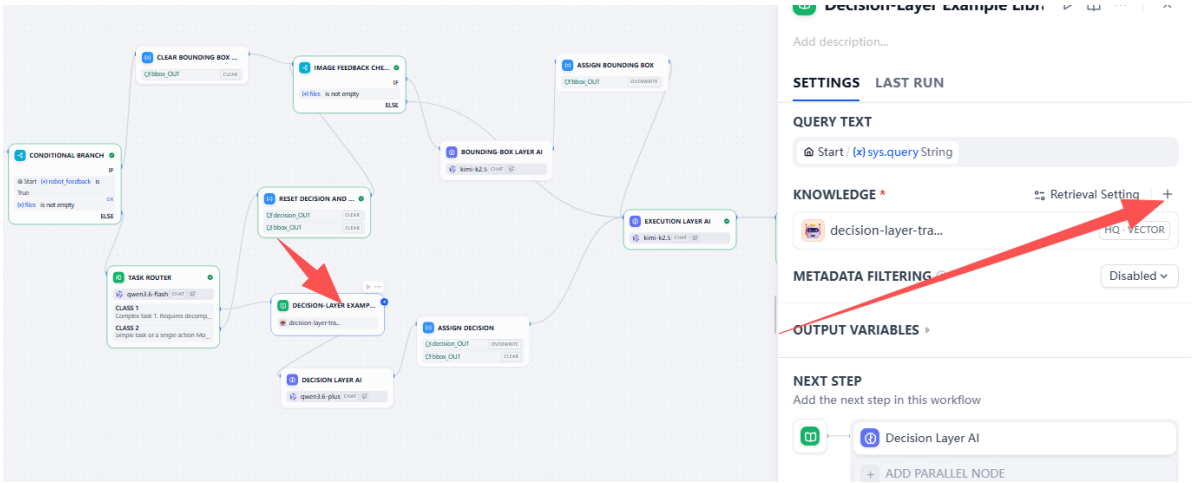
7. Return to the 【Studio】 interface and select the application corresponding to your camera model. Here, 【microros-v2-zh】 is used as an example.

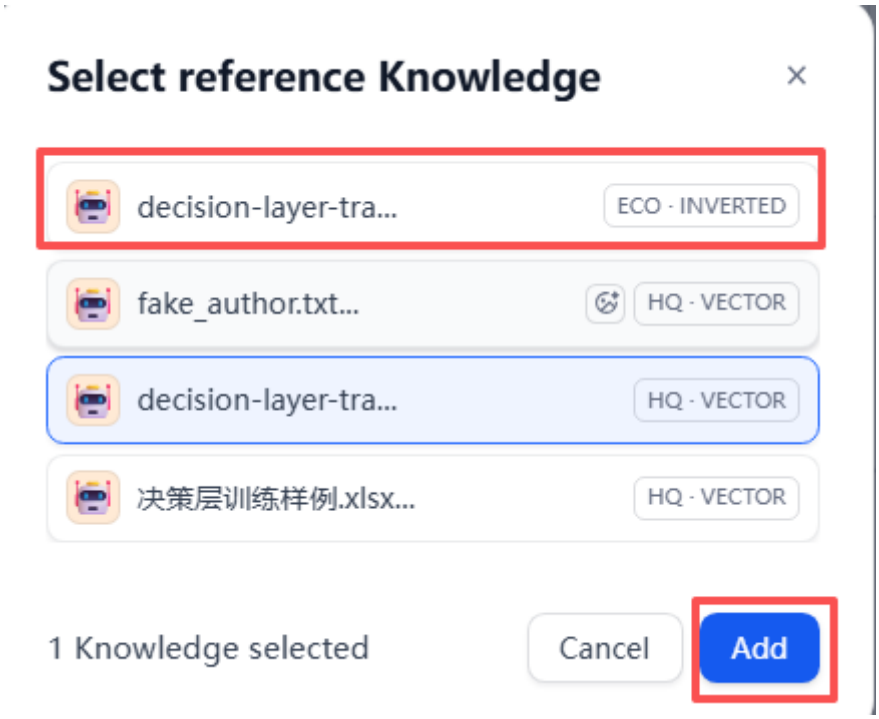


8. Select 【Task Routing】, and add your own defined intent expressions in the complex task category, for example, "I'm hungry", "I'm thirsty", and "I'm a little sleepy".



9. Select the 【Decision-Layer Sample Library】 knowledge retrieval node, and confirm that the newly added intent mapping knowledge base has been selected. If it has not been selected, click the Add button on the right to add the just-uploaded intent mapping knowledge base to this node.





10. You can click "Preview" in the upper-right corner, for example, send "I'm hungry" and observe whether it enters the complex task branch, and confirm whether the decision layer can output high-level steps consistent with the intent mapping.

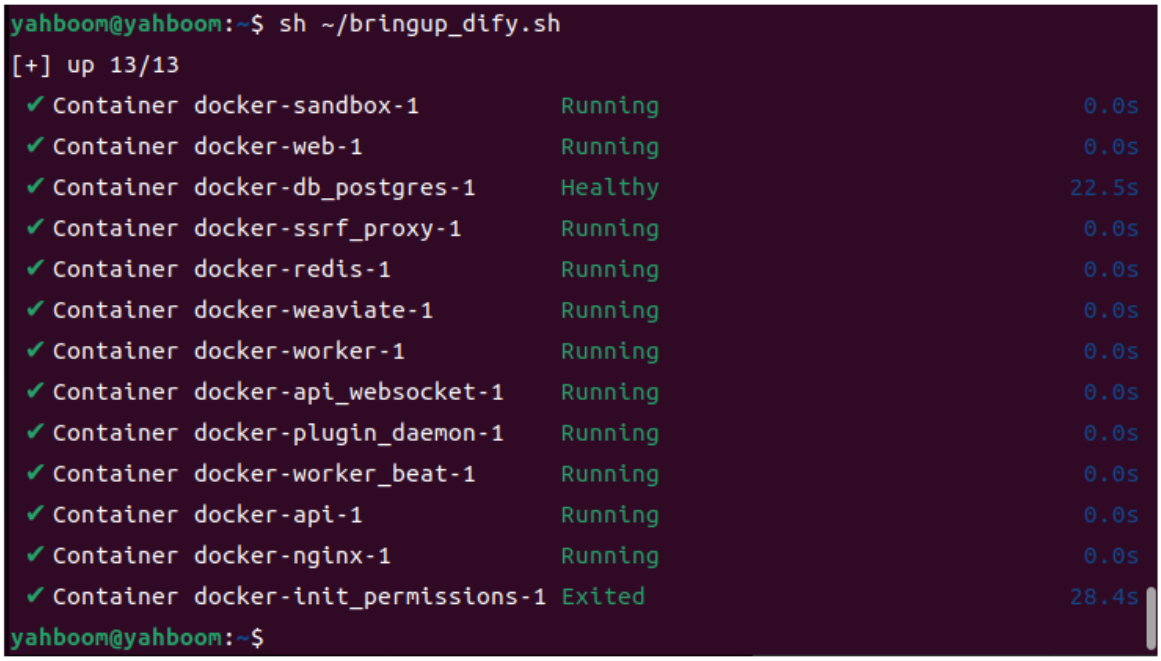
Finally, click 【Publish】 in the upper-right corner to re-publish the corresponding application.

4. Running the Case

4.1 Starting the Program

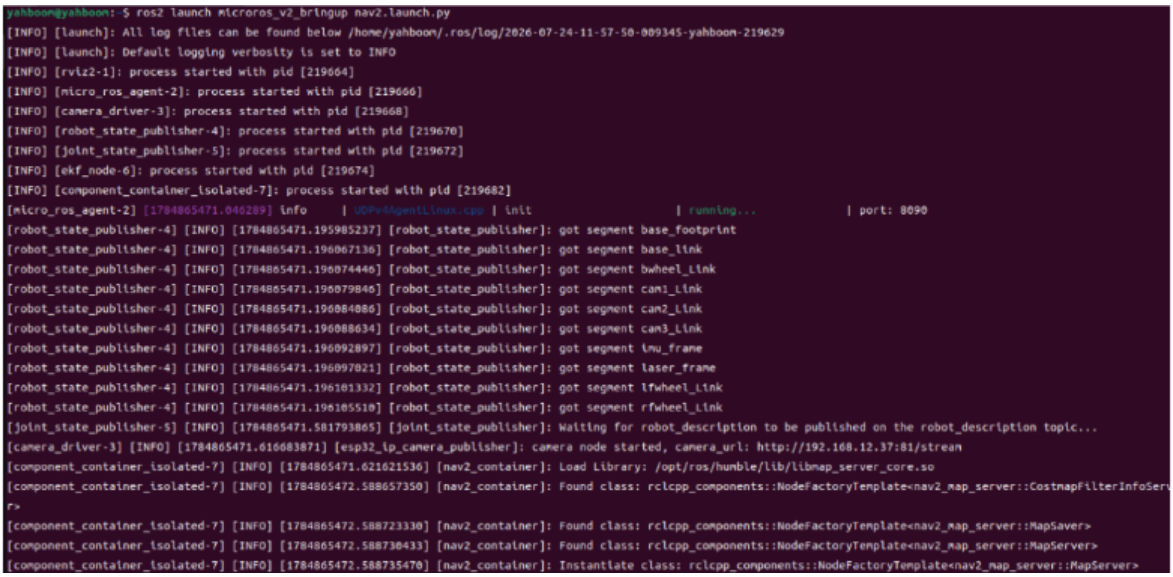
Open the virtual machine terminal, start the Dify environment (if it is already started, no need to start it again):

```
sh ~/bringup_dify.sh
```



Open the virtual machine terminal and enter the command to start the underlying proxy, camera topic, and Nav2 navigation:

```
ros2 launch microros_v2_bringup nav2.launch.py
```



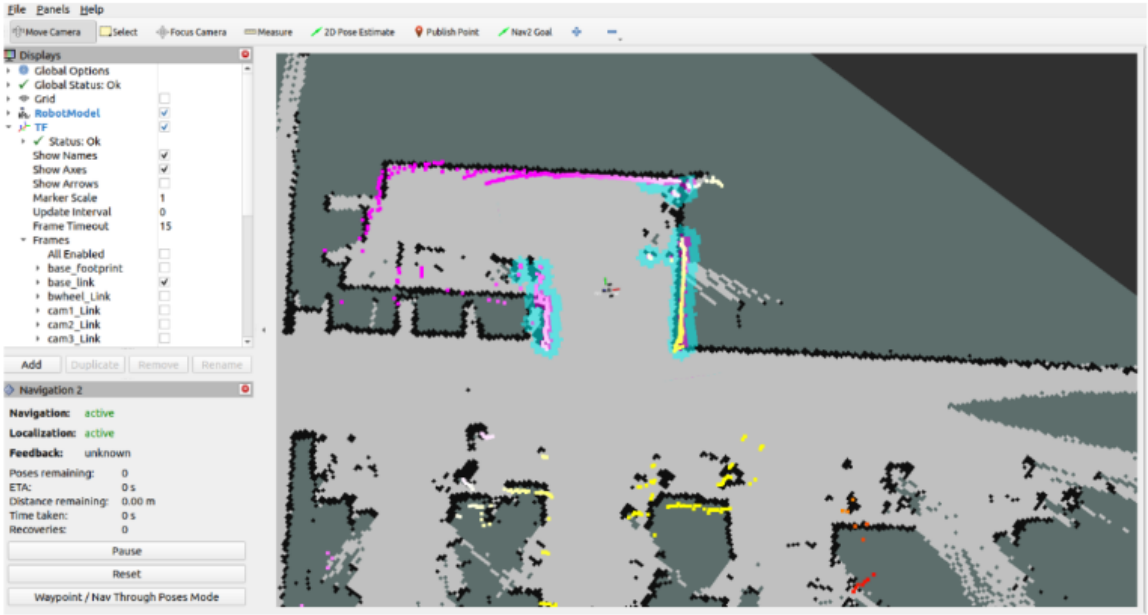
Open another virtual machine terminal and enter the command to start the large model agent node:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After initialization is complete, the following content will be displayed:

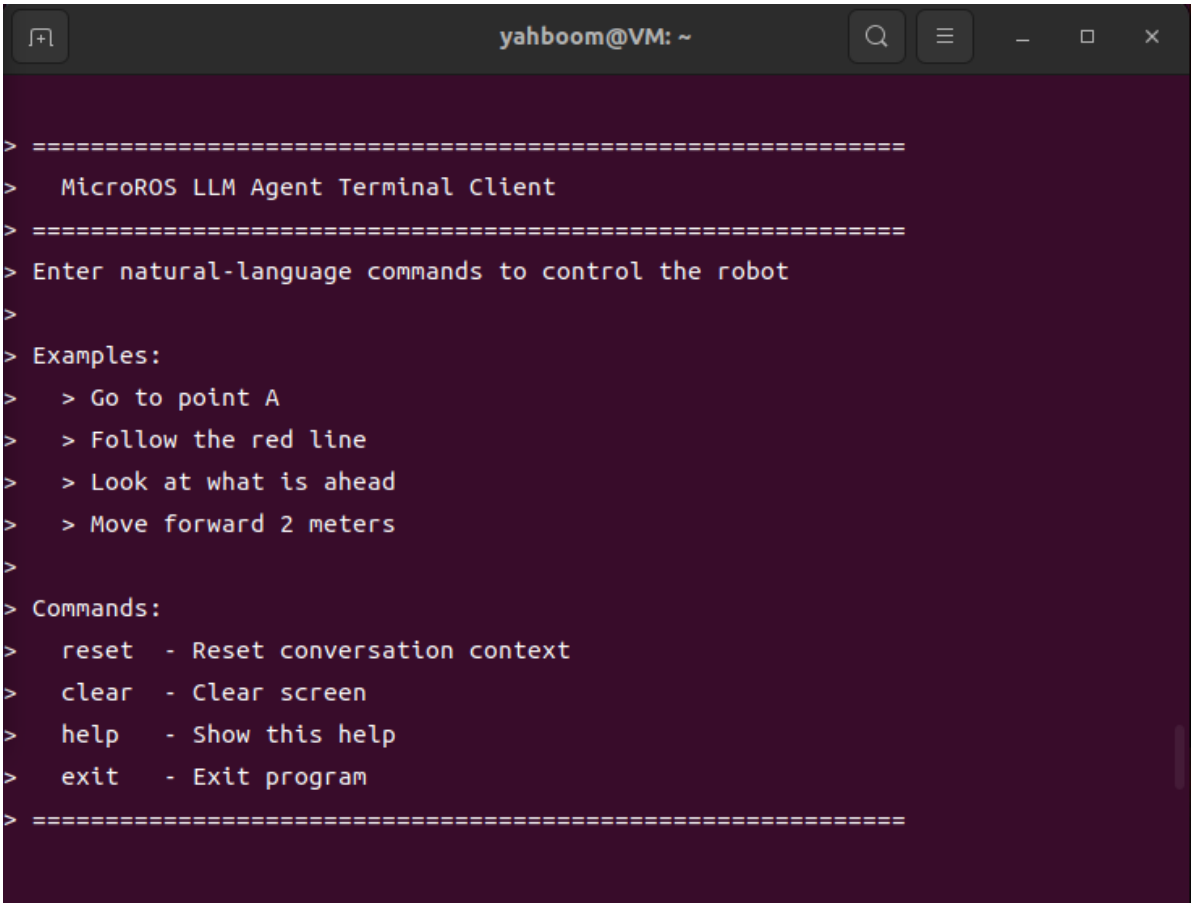
```
yahboom@VM: ~
G_MODE=False, LANGUAGE=en, CAR_TYPE=ptz
[llm_agent_node-1] [INFO] [1787128762.059479166] [llm_agent_node]: Initializing
LLM Agent Node...
[llm_agent_node-1] [INFO] [1787128762.070369334] [llm_agent_node]: [CONFIG] cmd_
vel_topic=/cmd_vel
[llm_agent_node-1] [INFO] [1787128762.077274270] [llm_agent_node]: [CONFIG] imag
e_cache_path=/home/yahboom/microros_ws/cache/llm_camera.jpg
[llm_agent_node-1] [INFO] [1787128762.078819208] [llm_agent_node]: [PTZ] startup
 initialization enabled: x=0.0°, y=-45.0°, count=20, interval=0.10s
[llm_agent_node-1] [INFO] [1787128762.106861388] [llm_agent_node]: [FOLLOW] scan
_topic=/scan, obstacle_guard=True, buzzer_topic=/beep, buzzer=True
[llm_agent_node-1] [INFO] [1787128762.107460365] [llm_agent_node]: [DIFY] Initia
lize client: base_url=http://localhost/v1, api_key=app-SAnX****Zt2E
[llm_agent_node-1] [INFO] [1787128762.109741022] [llm_agent_node]: [DIFY] Testin
g connection....
[llm_agent_node-1] [INFO] [1787128762.130633809] [llm_agent_node]: [TRACK_KCF] S
ubscribed /tracking_bbox, type=smart_tracker/msg/TrackBox
[llm_agent_node-1] [INFO] [1787128762.131212772] [llm_agent_node]: LLM Agent Nod
e initialized successfully
[llm_agent_node-1] [INFO] [1787128762.133300019] [llm_agent_node]: [DIFY] Connec
tion successful ✓
[llm_agent_node-1] [INFO] [1787128764.080740503] [llm_agent_node]: [PTZ] Servo i
nitialization complete: x=0.0°, y=-45.0°
```

Then follow the process of starting the navigation function for initial localization. After opening the RViz2 visualization interface, click **2D Pose Estimate** in the toolbar at the top, roughly mark the robot's position and orientation on the map. After initial localization, the preparation is complete.



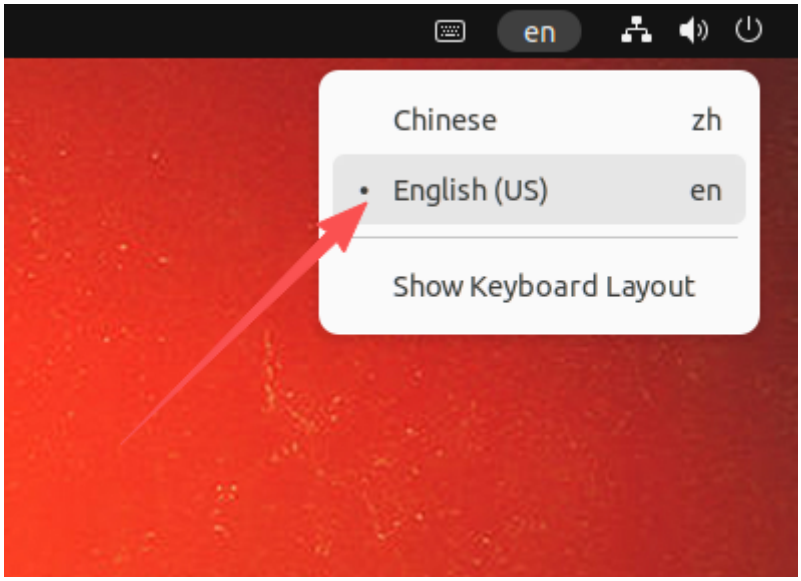
Then open another terminal and start the text dialogue client:

```
ros2 run multi_brains terminal_client
```



Use the Chinese input method:

- Click the input method icon in the upper-right corner of the virtual machine to switch to Chinese pinyin. You can also press the `shift` shortcut key in the terminal to switch.



4.2 Test Cases

Reference test cases are given here. Users can write their own dialogue commands:

- I feel a little thirsty.

4.2.1 Case 1

Enter "I feel a little thirsty." in the terminal. If the intent mapping knowledge base has a high-level task configured for "thirsty", for example, "go to the tea room/kitchen to get water or observe the drink location",

```
> [Sending] I'm feeling a little thirsty
> [Sent] New commands can be accepted. The latest command will take
priority.
>
>   [Progress] [Decision] Thinking...
>
>   [Progress] [Decision] Okay, navigating to the pantry now.
>
>   [Progress] Decision-making plan: 1. Navigate to Point A
>
>   [Progress] [Preparation] Adjusting the camera to look straight
ahead...
>   [Progress] [Action 1/1] navigation(A)
>
>   [Progress] [Navigation] Heading to Point A...
>
```

Then the execution-layer large model will execute according to the decision-layer steps:

```
llm_agent_node-1] [INFO] [1704882023.338633308] [llm_agent_node]: [PROGRESS] Decision layer plan: 1. Navigate to Point A
llm_agent_node-1] [INFO] [1704882023.339025700] [llm_agent_node]: [PROGRESS] [Preparation] Adjusting camera to face straight ahead...
llm_agent_node-1] [INFO] [1704882023.841612436] [llm_agent_node]: [PROGRESS] [Action 1/1] navigation(A)
llm_agent_node-1] [INFO] [1704882023.842059551] [llm_agent_node]: [NAVIGATION] Start common navigation: A
llm_agent_node-1] [INFO] [1704882023.842935262] [llm_agent_node]: [NAV] Target coordinate: pos=(-1.058, 0.621, 0.000), ori=(0.000, 0.000, -0.470, 0.882)
llm_agent_node-1] [INFO] [1704882023.849662594] [llm_agent_node]: [NAV] Waiting to arrive at 'A' (Timeout 300.0s)...
llm_agent_node-1] [INFO] [1704882023.849803047] [llm_agent_node]: [NAV] Navigation goal accepted: 'A'
llm_agent_node-1] [INFO] [1704882023.850200810] [llm_agent_node]: [PROGRESS] [Navigation] Moving towards Point A...
llm_agent_node-1] [INFO] [1704882040.446977929] [llm_agent_node]: [NAV] result_callback: status=4, result_type=NavigateToPose_Result
llm_agent_node-1] [INFO] [1704882040.447795513] [llm_agent_node]: [NAV] Navigation completed: status=4
llm_agent_node-1] [INFO] [1704882040.448157631] [llm_agent_node]: [NAV] Successfully arrived at 'A' ✓
llm_agent_node-1] [INFO] [1704882040.448602719] [llm_agent_node]: [PROGRESS] [Navigation] Arrived at Point A ✓
llm_agent_node-1] [INFO] [1704882040.449061419] [llm_agent_node]: [PROGRESS] [Action 1/1] navigation(A) Success ✓
llm_agent_node-1] [INFO] [1704882044.457743491] [llm_agent_node]: [PROGRESS] [Feedback Decision] Arrived at the pantry, now observing the surrounding environment
llm_agent_node-1] [INFO] [1704882044.458463476] [llm_agent_node]: [PROGRESS] Decision layer plan: 1. Take photos of surroundings from multiple directions and give feedback
llm_agent_node-1] [INFO] [1704882044.460985978] [llm_agent_node]: [PROGRESS] [Action 1/1] observe_surroundings()
llm_agent_node-1] [INFO] [1704882044.467778903] [llm_agent_node]: [OBSERVE] Start observing surrounding environment
llm_agent_node-1] [INFO] [1704882044.469268840] [llm_agent_node]: [PROGRESS] [Observe 1/3] Observing front...
llm_agent_node-1] [INFO] [1704882047.682917428] [llm_agent_node]: [PROGRESS] [Observe] Front: Waiting for new image timeout
llm_agent_node-1] [INFO] [1704882047.683351278] [llm_agent_node]: [PROGRESS] [Observe 2/3] Observing front-right...
llm_agent_node-1] [INFO] [1704882050.295290533] [llm_agent_node]: [PROGRESS] [Observe] Front-right: Acquired image from the corresponding orientation
llm_agent_node-1] [INFO] [1704882050.295751209] [llm_agent_node]: [PROGRESS] [Observe 3/3] Observing front-left...
llm_agent_node-1] [INFO] [1704882053.509204200] [llm_agent_node]: [PROGRESS] [Observe] Front-left: Waiting for new image timeout
llm_agent_node-1] [INFO] [1704882053.509706287] [llm_agent_node]: [OBSERVE] Surrounding image collection complete, total 1 direction
llm_agent_node-1] [INFO] [1704882053.510011258] [llm_agent_node]: [OBSERVE] Observation finished, gimbal returns to initial position
llm_agent_node-1] [INFO] [1704882054.013116417] [llm_agent_node]: [SERVO] Gimbal homing complete: x=0.0°, y=-45.0°
llm_agent_node-1] [INFO] [1704882054.013657825] [llm_agent_node]: [PROGRESS] [Action 1/1] observe_surroundings() Success ✓
llm_agent_node-1] [INFO] [1704882055.277632754] [llm_agent_node]: [DIFY] Feedback image 1/1 upload complete: file_id=37142c48-949c-4695-aecd-9aabf272fcc5, HTTP 201
llm_agent_node-1] [INFO] [1704882066.731691282] [llm_agent_node]: [Action] finishtask: Task cycle ended
llm_agent_node-1] [INFO] [1704882066.732467436] [llm_agent_node]: [RESULT] Arrived at the pantry and observed the surrounding environment, but did not find pantry faciliti
llm_agent_node-1] [INFO] [1704882066.732764635] [llm_agent_node]: [INPUT] == Command #8 processing complete, feedback rounds ≤12, total time: 50.67s ==
```

After the robot completes the task, it enters a waiting state, and the conversation history will continue to be retained. You can input `reset` in the terminal to reset the conversation context, or input a new natural language instruction to let the latest instruction take over the current task.

```
[llm_agent_node-1] [INFO] [1784867518.113324588] [llm_agent_node]: [INPUT] ── Received user input #3: '/
'reset'
[llm_agent_node-1] [INFO] [1784867518.114754858] [llm_agent_node]: [PROGRESS] [DECISION] Thinking...
[llm_agent_node-1] [INFO] [1784867522.880420933] [llm_agent_node]: [ACTION] finishtask: Task cycle ended
[llm_agent_node-1] [INFO] [1784867522.881048600] [llm_agent_node]: [RESULT] System reset, awaiting new
instructions
[llm_agent_node-1] [INFO] [1784867522.881456002] [llm_agent_node]: [INPUT] --- Instruction #3 processed,
feedback turn <12, totatal time: 4.77s ---
```

5. Source Code Analysis

The key new point in this chapter is not in the new ROS action source code, but in the **Dify intent mapping knowledge base and task routing**. The ROS side still reuses the navigation, visual understanding, and feedback loop capabilities covered in Chapter 6.

Current relevant source code and configuration paths:

```
~/microros_ws/src/multi_brains/multi_brains/llm_agent_node.py
~/microros_ws/src/multi_brains/multi_brains/services/dify_coordinator.py
~/microros_ws/src/multi_brains/multi_brains/controllers/navigation_controller.py
~/microros_ws/src/multi_brains/config/llm_agent_config.yaml
~/microros_ws/src/multi_brains/config/map_mapping.yaml
```

5.1 Where the New Capability Is

"Intent inference" does not add new action functions such as `hunger()` or `thirsty()`, but adds a layer of personal semantic mapping to Dify. For example, when the user says "I'm a little thirsty", the knowledge base supplements it as "need to go to the tea room/kitchen to observe whether there is water or a drink", and then the decision layer decomposes this goal into steps such as navigation, observation, and return.

Three Dify configuration points need attention in the workflow:

- **Task routing**: classifies personal expressions such as "I'm hungry", "I'm thirsty", and "I'm a little sleepy" into the complex task branch.

- **Decision-layer sample library/knowledge retrieval node**: selects the uploaded intent mapping knowledge base so that the decision layer can retrieve the task description corresponding to personal intents.
- **Conversation variable** `map_mapping`: maps location names such as "tea room" and "office" to navigation target IDs such as `A` and `B`.

If any of these three is missing, the robot will not be able to reliably derive executable actions from "implicit intents".

5.2 Core Flow

```
User inputs "I'm currently in the office area, and I feel a little thirsty"
  -> Dify task routing determines it as a complex task/intent inference
  -> Knowledge retrieval node recalls the "thirsty"-related task description from
the intent mapping knowledge base
  -> Decision layer generates high-level steps combined with map_mapping
  -> Execution layer outputs only one action JSON per round
  -> llm_agent_node.py parses the action field and distributes it through
action_registry
  -> NavigationController executes get_current_pose() / navigation(target)
  -> seewhat() uploads the current frame to Dify as robot feedback
  -> Dify continues to the next step based on navigation or image feedback
  -> finishtask() ends the current task cycle
```

The feedback loop of `llm_agent_node.py` reads the execution result and then uses "robot feedback: executing {actions} completed/failed" to continue requesting Dify. Complex tasks do not execute multiple actions at once, but instead follow "decide one round, execute one action, and after feedback, decide the next round".